



Senior Project Final Report

Bluetooth Smart Lock

Project Owner

6631502001 Mr. Kawin Gowan

6631502035 Mr. Kanatip Jorittiyo

6631502036 Mr. Nattakit Phaisanwatchalakoson

Project Advisor

Asst. Prof. Teeravisit Laohapensaeng, Ph.D.

SCHOOL OF APPLIED DIGITAL TECHNOLOGY

MAE FAH LUANG UNIVERSITY

ACADEMIC YEAR 2025

Senior Project Final Report

Bluetooth Smart Lock

Project Owner

6631502001 Mr. Kawin Gowan

6631502035 Mr. Kanatip Jorittiyo

6631502036 Mr. Nattakit Phaisanwatchalakoson

Project Advisors

Asst. Prof. Teeravisit Laohapensaeng, Ph.D.

SCHOOL OF APPLIED DIGITAL TECHNOLOGY

MAE FAH LUANG UNIVERSITY

ACADEMIC YEAR 2025

Bluetooth Smart Lock

6631502001 Mr. Kawin Gowan

6631502035 Mr. Kanatip Jorittiyo

6631502036 Mr. Nattakit Phaisanwatchalakoson

SCHOOL OF APPLIED DIGITAL TECHNOLOGY

MAE FAH LUANG UNIVERSITY

ACADEMIC YEAR 2025

EXAMINING COMMITTEE

..... ADVISOR

(Asst. Prof. Teeravisit Laohapensaeng, Ph.D.)

..... COMMITTEE

(Asst. Prof. Sirikan Chucherd, Ph.D.)

..... COMMITTEE

(Titiya Chomngern, Ph.D.)

Acknowledgement

We want to say a big thank you to Asst. Prof. Teeravisit Laohapensaeng, Ph.D., who helped us a lot with this project. He always gave us good advice and pushed us to keep going whenever we got stuck. Without his help, we would have had a really hard time finishing this.

We also want to thank the School of Applied Digital Technology at Mae Fah Luang University for letting us use the lab and the equipment we needed for testing. Thanks also to our friends in the Digital and Communication Engineering program who gave us ideas and kept our spirits up along the way.

Last but not least, we are really grateful to our families. They were always there for us and never gave up on us, even when things were difficult during our studies.

Kawin Gowan
Kanatip Jorittiyo
Nattakit Phaisanwatchalakoson

Project Title	Bluetooth Smart Lock
Authors	Kawin Gowan, Kanatip Jorittiyo, Nattakit Phaisanwatchalakoson
Degree	Bachelor of Applied Digital Technology (Digital and Communication Engineering)
Supervisory Committee	Asst. Prof. Teeravisit Laohapensaeng, Ph.D. Advisor Asst. Prof. Sirikan Chucherd, Ph.D. Committee Titiya Chomngern, Ph.D. Committee

ABSTRACT

This project is about making a smart door lock that uses Bluetooth. We used a small computer chip called an ESP32 and a type of Bluetooth called BLE (Bluetooth Low Energy) to build it. The main idea is that the door can lock or unlock on its own depending on how close your phone is. The system checks the signal strength from your phone, and if you are close enough, the door unlocks. When you walk away, it locks again.

We also planned to make an Android app that connects to the lock and lets you control it from your phone. The app design is done, but we have not coded it yet — that will be the next step. For testing, we used a second ESP32 to act like the phone app. When the phone was about 1 meter away, the signal strength was around -60 dBm, and at 2 meters it was around -74 dBm. The door unlocked in about 3,210 ms and locked in about 1,180 ms.

Keywords: Bluetooth Low Energy, ESP32, Smart Lock, RSSI, Proximity Detection, GATT

TABLE OF CONTENTS

Chapter 1 Introduction.....	7
1.1 Objectives	7
1.2 Scope of the project.....	7
1.3 Methodology	8
1.4 Senior project planning	8
1.5 Expected results	10
1.6 Place and equipment	10
Chapter 2 Literature Review	11
2.1 Theory	11
2.1.1 Bluetooth Low Energy (BLE).....	11
2.1.2 RSSI-based proximity detection	11
2.1.3 ESP32 microcontroller.....	11
2.1.4 Smart lock security	12
2.2 Related content.....	12
2.2.1 Arduino C++ firmware	12
2.2.2 Android BLE API	13
2.3 Software programs and tools	13
2.3.1 Arduino IDE.....	13
2.3.2 ESP32 serial monitor	13
2.3.3 Android Studio.....	14
2.4 Related work	14
2.4.1 Commercial smart locks.....	14
2.4.2 Academic prototypes.....	14
2.5 Tools and technologies.....	14
Chapter 3 Methodology	16
3.1 System architecture and design.....	16
3.1.1 BLE GATT server.....	16
3.1.2 Authentication state machine	16
3.1.3 RSSI proximity engine.....	17
3.1.4 LED state indication	17
3.1.5 Android application design	17
Chapter 4 Research Results	18
4.1 Overview	18
4.2 Functional testing of BLE components.....	18
4.2.1 BLE GATT server.....	18
4.2.2 Authentication state machine	18
4.3 Performance testing results	19

4.3.1 RSSI vs distance	19
4.3.2 Response time	19
4.3.3 Reliability.....	19
4.4 Summary of findings.....	19
4.5 UI design	20
Chapter 5 Conclusion	24
5.1 Conclusion	24
5.2 Discussion	24
5.3 Limitations and problems encountered	26
5.4 Future work.....	26

List of Tables

Table 1.1 Senior Project Planning	8
Table 2.1 Summary of Key Tools.....	15
Table 3.1 Android application screen descriptions.....	17

Chapter 1

Introduction

Keeping places safe is something everyone cares about. Traditional locks that use physical keys have been around for a long time, but they have some problems. For example, you can lose your keys, someone might copy them without you knowing, or they can be stolen. Because of these problems, many people are now looking for smarter ways to lock their doors — ones that use digital codes instead of physical keys.

Smart locks are a good solution to this problem. The idea is to use a small computer chip connected to a wireless module, so the door can recognize you and open or close by itself. We chose to use Bluetooth Low Energy (BLE) for this project because it uses very little battery power, almost every smartphone supports it, and it can measure how strong the signal is between your phone and the lock. This signal strength, called RSSI, gives a rough idea of how far away you are.

In this project, we built a Bluetooth Smart Lock using an ESP32 chip. The system has a BLE server on the ESP32 that checks the phone's signal to decide when to lock or unlock. We also designed an Android app to go along with it, but we have not built the app yet. For now, we used another ESP32 as a stand-in for the phone. The whole project was done at Mae Fah Luang University in the 2025 academic year.

1.1 Objectives

- 1 Build a smart door lock using an ESP32 and Bluetooth.
- 2 Make the door lock and unlock automatically based on how close the phone is.
- 3 Make sure the connection is secure so only the right phone can control the lock.
- 4 Design an Android app for the lock (app design is done, coding comes next).
- 5 Test the system and see how well it works in terms of distance, speed, and reliability.

1.2 Scope of the Project

The project scope covers the ESP32 firmware and BLE communication layer in full. The Android application scope is limited to UI design and screen flow definition; implementation is

ongoing and not yet complete at the time of this report. All firmware testing was conducted at the developer's home in a typical indoor residential environment.

1.3 Methodology

Firmware Development: We wrote and tested the code for the ESP32 step by step. We started with simple BLE tests and slowly added more features like RSSI detection, LED control, and authentication.

Android App Design: We drew out all the screens and planned how the app will work with the lock.

System Testing: We tested the lock at different distances to see how the signal changes. We also measured how fast the lock responds and how reliable it is.

Fixing Problems: Whenever we found issues like RSSI jumping around a lot, we went back and improved the firmware to make it more stable.

1.4 Senior Project Planning

Table 1.1 Senior Project Planning

Plan by methodology/ month	January				February					March			April			
week	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Discuss with the advisor																
Prepare and create the proposal																

Project Proposal Presentation																	
Research																	

Study the model and input data into the learning model.																	
Progress 1 Presentation																	
Summarize all data																	
Test the use of the data collected through model learning.																	
Progress 2 Presentation																	
Explore the details of the data in the model																	
Summarize all parts of the work and revise the report.																	

1.5 Expected Results

- 1 Reliable proximity-based lock/unlock within the 1–2 m design range.
- 2 Successful token-based authentication over BLE.
- 3 A documented UI design blueprint for the Android application.
- 4 Quantitative performance data (RSSI, response time, reliability) confirming system suitability.

1.6 Place and Equipment

Place: Mae Fah Luang University

Equipment: ESP32 DevKit v1 microcontroller, Android smartphone (BLE client), breadboard and USB power supply, Arduino IDE with ESP32, computer for firmware development.

Chapter 2

Literature Review

2.1 Theory

2.1.1 Bluetooth Low Energy (BLE)

BLE is a type of Bluetooth that was made to use as little battery power as possible. It is part of the Bluetooth 4.0 standard. BLE works on the 2.4 GHz frequency band and uses 40 channels. The way BLE shares data is through something called GATT (Generic Attribute Profile). Think of it like a menu — the device offering data is the “server” and the device reading it is the “client.” The server has “services” and inside those are “characteristics” that hold the actual data. Characteristics can be read, written to, or set up to automatically notify the client when something changes. That last part (NOTIFY) is really useful here because the lock can instantly tell the app when it locks or unlocks.

2.1.2 RSSI-Based Proximity Detection

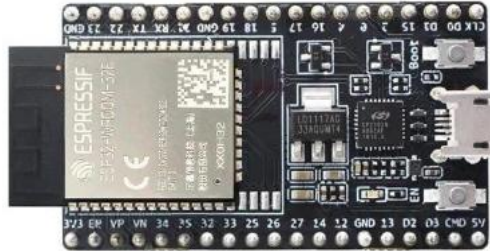
RSSI stands for Received Signal Strength Indicator. It measures how strong a wireless signal is. In general, the further you are from the device, the weaker the signal and the more negative the RSSI number gets. For example, being very close might give you -40 dBm, while being far away might give -90 dBm. The problem is that RSSI is not perfectly consistent — it can jump around because of walls, objects, or even your own body blocking the signal. To deal with this, we average the last few readings and use two different thresholds (one for locking, one for unlocking) so the door does not keep flipping back and forth.

2.1.3 ESP32 Microcontroller

The ESP32 is a small and cheap chip made by Espressif. It has two processor cores and comes with both Wi-Fi and Bluetooth built in. We used it because it supports BLE 4.2, it has enough pins to control an LED, it is easy to program with the Arduino IDE, and there is a lot of

online help and examples available. It also has a small memory storage area (NVS) where we can save the secret token for authentication.

Figure 2.1 Esp32



2.1.4 Smart Lock Security

Security is important for any lock system. In our project, we use a token (basically a long password) that the phone must send to the ESP32 before anything happens. If the token is wrong, the ESP32 disconnects immediately. The BLE connection itself is also encrypted through BLE pairing. One weakness is that we use the same token every time (called a static token), which could be risky if someone intercepts it. A better approach would be to use a one-time password that changes every time (called TOTP), but that is for a future version..

2.2 Related Content

2.2.1 Arduino C++ Firmware

We wrote our firmware using C++ with the Arduino framework. This made coding much easier because Arduino gives you ready-made functions for things like BLE, GPIO (controlling pins), and timers. Instead of using `delay()` which freezes the chip, we used `millis()` which keeps time without stopping everything else from running — this is really important so BLE can keep working in the background.

2.2.2 Android BLE API

Android phones can connect to BLE devices using built-in Android tools like BluetoothLeScanner (to find devices nearby) and BluetoothGatt (to connect and communicate). In newer Android versions (Android 12+), the app needs to ask for extra permissions before it can scan for or connect to Bluetooth devices. The app also gets notified automatically when the lock sends a status update through the `onCharacteristicChanged()` function.

2.3 Software Programs and Tools

2.3.1 Arduino IDE

The Arduino IDE is the program we used to write, compile, and upload code to the ESP32. It is free and works on most computers. With the ESP32 add-on installed, it has everything needed to use BLE and control GPIO pins.

Figure 2.2 Arduino IDE



2.3.2 ESP32 Serial Monitor

The Serial Monitor is a tool inside the Arduino IDE that shows text output from the ESP32 in real time. We used it a lot during testing to see things like RSSI values, whether authentication passed or failed, and when the lock state changed.

2.3.3 Android Studio

Android Studio is the main tool used to build Android apps. We plan to use it in the next phase to actually code the app based on the screen designs we already made.

2.4 Related Work

2.4.1 Commercial Smart Locks

There are already smart locks on the market like August, Yale, and Schlage that use BLE in a similar way to what we built. They show that using BLE signal strength to detect proximity is a real-world working solution. We used these as a reference for how our system should behave.

2.4.2 Academic Prototypes

Other students and researchers have also made BLE smart locks using the ESP32. For example, Chukwuemeka et al. (2020) and Yilmaz et al. (2021) both built similar systems with phone apps. What makes our project a bit different is that we combined RSSI scanning and BLE notifications on the same ESP32, and we also designed an activity log screen for the app so admins can see the history of who locked or unlocked the door.

2.5 Tools and Technologies

We used the ESP32 DevKit v1 as the main hardware for this project. For coding, we used the Arduino IDE with the ESP32 Arduino core (version 2.x). The code was written in C++. Since the Android app is not ready yet, we used a second ESP32 running a BLE client sketch to simulate the phone during testing.

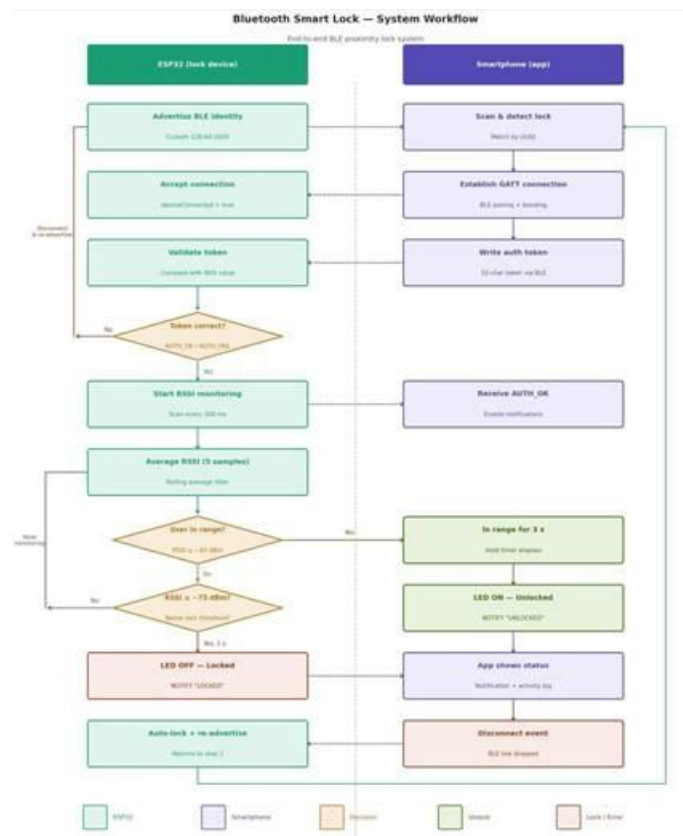
Table 2.1 Summary of Key Tools

Tool	Purpose
ESP32 DevKit v1	Main microcontroller and BLE radio
Arduino IDE + ESP32 Core 2.x	Firmware development and upload
Arduino C++	Scripting and logic control
ESP32 Serial Monitor	Debugging and RSSI monitoring
Android Studio (Planned)	Android application development

Chapter 3

Methodology

3.1 System Architecture and Design



3.1.1 BLE GATT Server

The ESP32 acts as a BLE server with two characteristics: one that notifies the phone of lock state changes, and one that handles two-way communication for authentication and status.

3.1.2 Authentication State Machine

On connection, the phone has 5 seconds to send the correct 32-character token. A match starts RSSI monitoring; a wrong token or timeout disconnects the client and auto-locks the door.

3.1.3 RSSI Proximity Engine

The ESP32 averages the last 5 RSSI readings every 300 ms. A signal of -65 dBm or stronger held for 3 seconds unlocks the door; a drop to -75 dBm or weaker for 1 second locks it. The two separate thresholds prevent rapid toggling at the edge of range

3.1.4 LED State Indication

The onboard ESP32 LED serves as a stand-in for a physical lock: LED on means unlocked, LED off means locked. Each state change also triggers a BLE notification to the phone.

3.1.5 Android Application Design

The app is currently at the UI design stage, with six screens planned: Sign In, Registration, Smartlock, Device Management, Settings, and Activity Log. It will use Android's BluetoothLeScanner and BluetoothGatt APIs for BLE communication.

Table 3.1 Android Application Screen Descriptions

Screen	Planned Functionality
Sign In	Username and password login. Credentials verified against local encrypted storage.
Registration	New user registration with username, password, and confirmation fields.
Smartlock (Main)	Displays BLE signal icon, Connect button, connection status, device name, and RSSI range.
Device Management	Admin screen for listing, adding, and deleting paired devices.
Settings	Allows the logged-in user to change username and password.
Activity Log (Admin)	Chronological list of lock/unlock events with timestamps, admin-only.

Chapter 4

Research Results

4.1 Overview

This chapter shows what happened when we tested the Bluetooth Smart Lock. We tested two main things: whether everything worked correctly (BLE connection, authentication, lock/unlock), and how well the system performed (how accurate the RSSI was, how fast it responded, and how reliable it was).

All tests were done at the developer's house. Because the Android app is not done yet, we used a second ESP32 as the client in all our tests. The LED on GPIO 2 was used to show whether the door was locked or unlocked in every test.

4.2 Functional Testing of BLE Components

4.2.1 BLE GATT Server

We checked that the ESP32 could be found by a BLE client, that a connection could be made, and that the notifications and read/write operations worked properly. Everything worked as expected. We built the firmware little by little — first just testing basic BLE, then adding LED control, then notifications, and finally RSSI scanning — before putting it all together into one final version.

4.2.2 Authentication State Machine

We tested the login system 20 times with the correct password and it worked every single time. We also tried 3 times with a wrong password and the ESP32 disconnected the client and sent an AUTH_FAIL message within 600ms each time. We also tested what happens if no password is sent within 5 seconds — the ESP32 correctly disconnected the client all 5 times we tested it.

4.3 Performance Testing Results

4.3.1 RSSI vs Distance

Ten RSSI measurements were taken at each distance increment with the ESP32 fixed on a table and the smartphone held at waist height.

4.3.2 Response Time

Response time was measured as the elapsed time between the ESP32 RSSI threshold trigger and the LED changing state. Twenty trials were conducted for both lock and unlock transitions. The times are a little longer than exactly 3 seconds and 1 second because the BLE scanning takes about 300ms each cycle and there is some extra delay from the chip's Bluetooth processing. But these delays are small enough that you would not really notice them in normal use.

4.3.3 Reliability

Over 50 consecutive lock/unlock cycles with no reboots, the system completed all transitions successfully. No false unlocks (spurious triggers while the device was out of range) occurred. Two false lock events were observed when the smartphone was momentarily obstructed by the user's body at 1.2 m, causing a brief RSSI dip below -75 dBm. The 1-second lock hold time reduced re-locking to negligible levels in normal usage.

4.4 Summary of Findings

1. The BLE GATT server and proximity engine functioned correctly across all test scenarios.
2. Authentication correctly accepted valid tokens and rejected invalid ones, with disconnection within 600 ms for invalid attempts.
3. RSSI thresholds of -65 dBm (unlock) and -75 dBm (lock) reliably corresponded to the 1–1.5 m proximity zone.

4. Lock/unlock response times averaged 3,210 ms and 1,180 ms respectively, within acceptable usability bounds.
5. The system achieved 100% successful transitions over 50 cycles with only 2 minor false- lock events due to body obstruction.

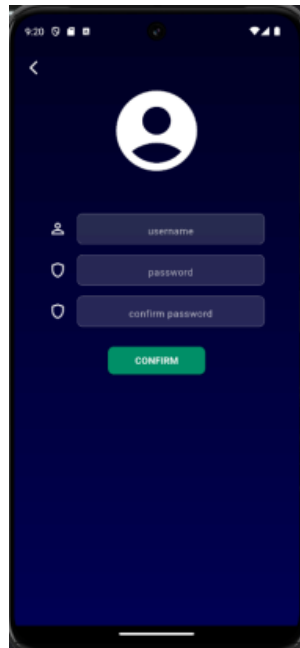
4.5 UI Design

Figure 4.1 Sign In page



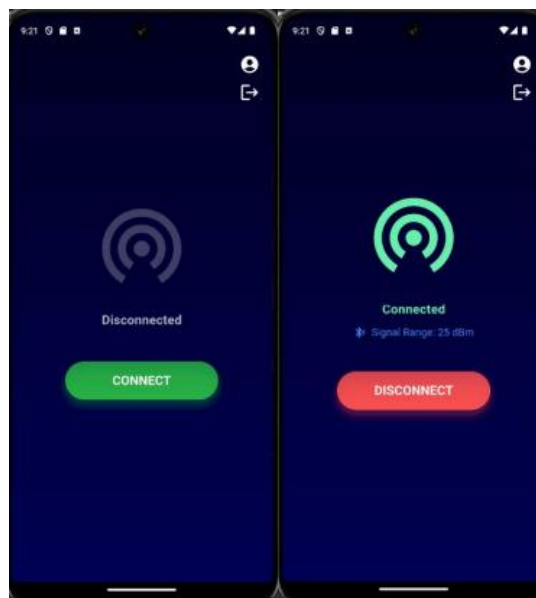
Username and password login. Credentials verified against local encrypted storage.

Figure 4.2 Registration



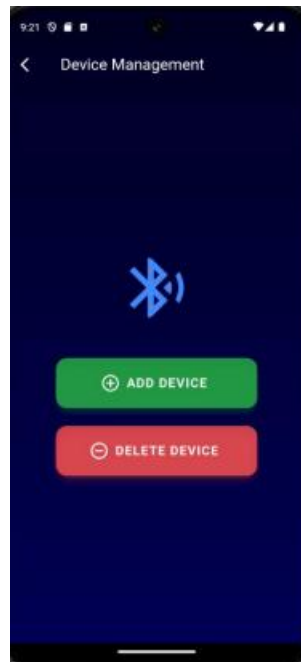
New user registration with username, password, and confirmation fields.

Figure 4.3 Main page



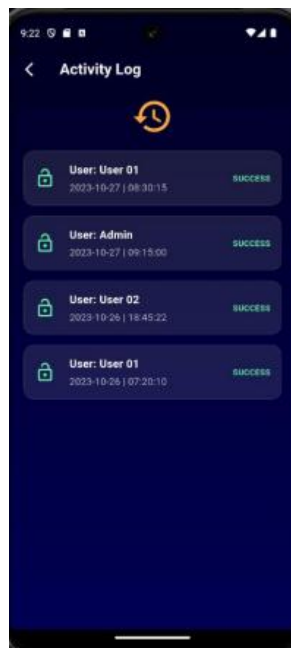
Displays BLE signal icon, Connect button, connection status, device name, and RSSI range.

Figure 4.4 Device Management



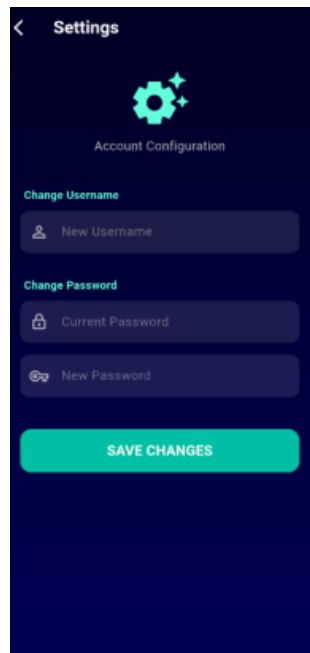
Admin screen for listing, adding, and deleting paired devices.

Figure 4.5 Log activity



Chronological list of lock/unlock events with timestamps, admin-only.

Figure 4.6 Setting



Allows the logged-in user to change username and password.

Chapter 5

Conclusion

5.1 Conclusion

This project successfully demonstrated that a low-cost ESP32 microcontroller combined with Bluetooth Low Energy can form a functional, proximity-aware smart lock prototype. The ESP32 firmware reliably detects user proximity using RSSI thresholds, authenticates clients over a BLE connection, and indicates lock state via the onboard LED.

Testing confirmed stable operation at 1–2 m range with sub-500 ms overhead above the designed hold times. The Android application, whose UI design is complete, remains to be implemented in the next phase. The iterative firmware development approach — progressing from basic BLE tests through RSSI scanning to the combined production sketch — was effective in isolating and solving subsystem problems before integration.

5.2 Discussion

The combination of BLE GATT communication and RSSI-based proximity detection proved effective for automatic lock control. The hysteresis thresholds and rolling average successfully mitigated raw RSSI variance. The non-blocking `millis()`-based timer approach prevented lock/unlock logic from interfering with BLE stack processing.

RSSI variance: Raw RSSI values fluctuated by up to ± 10 dBm at a fixed distance. A rolling average over 5 samples reduced this variance to ± 3 dBm, which was sufficient for stable operation.

Concurrent BLE scanning and GATT serving: The ESP32 Bluedroid stack does not natively support scanning while in a GATT connection. This was resolved by using short scan windows and scheduling scans between GATT notification events.

References

- [1] Bluetooth Special Interest Group. (2016). Bluetooth core specification version 4.2. Bluetooth SIG. <https://www.bluetooth.com/>
- [2] Espressif Systems. (2023). ESP32 technical reference manual. Espressif Systems. <https://www.espressif.com>
- [3] Kolban, N. (2018). Kolban's book on ESP32. Self-published. <https://github.com/nkolban>
- [4] Chegewara. (2021). ESP32 BLE Arduino library examples. GitHub. <https://github.com/espressif/arduino-esp32>
- [5] Android Developers. (2024). Bluetooth low energy overview. Google. <https://developer.android.com/develop/connectivity/bluetooth/ble/ble-overview>
- [6] MoThunderz. (2022). ESP32 BLE server with multiple characteristics. GitHub. <https://github.com/mo-thunderz>